

RA5 PYJINJA2+FASTAPI

Instrucciones

Objetivo: desarrollar un **informe web de incidencias** con FastAPI, Jinja2 y Chart.js. Una empresa necesita consultar sus incidencias, aplicar filtros y ver un resumen de los resultados.

Parte del [repositorio de clase py-jinja](#) y de su [enunciado original](#). El código de ejemplo genera un informe de **tareas**: debes adaptarlo al caso de **incidencias** descrito aquí.

1. Prepara los datos

Puedes definir una lista de diccionarios en Python o cargar un archivo JSON sencillo. Cada incidencia debe incluir:

- **id y título:** identificador y descripción breve.
- **categoría:** red, hardware o software.
- **gravedad:** un número entero entre 1 y 5.
- **resuelta:** un valor booleano que indique si está solucionada.

Incluye casos de distintas categorías y gravedades, con incidencias resueltas y pendientes. Puedes añadir otros campos, como la fecha. No hace falta una base de datos.

2. Implementa la ruta y los filtros

La ruta GET `/informe` debe devolver una página HTML generada en el servidor mediante Jinja2. Añade un formulario que envíe estos parámetros por GET:

- **categoría:** filtra por una categoría concreta; la opción «Todas» no limita la categoría.
- **min_gravedad:** muestra las incidencias cuya gravedad sea igual o superior al valor indicado, entre 1 y 5.

Ejemplo: `/informe?categoria=software&min_gravedad=3` debe mostrar solo incidencias de software con gravedad 3, 4 o 5. Sin filtros, muestra todas las incidencias.

Conserva visibles los filtros seleccionados y valida sus valores en el servidor. Describe qué ocurre si se recibe una categoría desconocida o una gravedad fuera del rango.

3. Construye el informe

Usa `templates/base.html` para la estructura común y `templates/informe.html` como plantilla del informe que hereda de ella.

- **Resumen:** número de incidencias, número de resueltas y porcentaje de resolución.
- **Tabla:** detalle de las incidencias que cumplen los filtros, con su identificación, categoría, gravedad y estado.
- **Gráfico de Chart.js:** número de incidencias por categoría.

Calcula el resumen y los conteos en Python sobre **el mismo conjunto filtrado** que muestra la tabla. Si no hay resultados, muestra un mensaje, totales a cero y un porcentaje de 0 %, sin dividir entre cero.

Pasa los datos a Jinja2 y utiliza bucles y condiciones para construir la vista. Para transferir los datos del gráfico a JavaScript, utiliza el filtro `tojson`. El formulario recarga el informe desde el servidor; no necesitas crear una API adicional para consultarla con `fetch`.

4. Realiza una modificación propia

Es obligatorio modificar la plantilla al menos una vez. Elige una de las opciones del enunciado original:

- Añadir una nueva sección que aporte información al informe.
- Añadir una nueva columna relevante a la tabla.
- Añadir un segundo gráfico; por ejemplo, la distribución por gravedad.

Explica qué has añadido, de dónde salen sus datos y qué aporta al usuario. Si muestra totales o agrupaciones, también debe respetar los filtros activos.

5. Comprueba el funcionamiento y el despliegue

1. Abre el informe sin filtros y contrasta los totales con tus datos.
2. Prueba cada filtro por separado y los dos combinados.
3. Comprueba un caso sin resultados y los valores mínimo y máximo de gravedad.
4. Verifica que tabla, resumen y gráficos coinciden tras cada cambio.
5. Prueba el despliegue con los archivos Docker y Compose del repositorio de clase. Documenta el comando, la dirección de acceso y el resultado.

Si cargas Chart.js desde una CDN, indica que los gráficos requieren conexión a Internet. Describe cualquier cambio necesario para ejecutar el proyecto con tus versiones de las dependencias.

Entrega

Sube el proyecto a tu repositorio de GitHub e incluye:

- `main.py`, `templates/base.html` y `templates/informe.html`, junto con los datos y archivos necesarios para ejecutarlo.
- Dependencias e instrucciones de arranque y despliegue.
- Una captura del informe con algún filtro aplicado.
- Una memoria breve en el `README.md`: estructura del informe, filtros utilizados, cálculos de totales, modificación de la plantilla y comprobaciones realizadas.

Debes poder explicar cómo pasan los datos desde Python a la plantilla y cómo afectan los filtros a cada parte del informe.

Consulta: [plantillas en FastAPI](#) y [filtro tojson de Jinja2](#).

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



Proyecto de partida · py-jinja

Repositorio original de clase · incluye ENUNCIADO.md y archivos para el despliegue

